

Lecture 2

ESSLLI Course — *Introduction to Homotopy Type Theory / Univalent Foundations*

Tom de Jong

Additional reading

The citations refer to the file `Resources.pdf`. We roughly cover [Rij25, Sections 1.1–6.2]. See [Esc19, Sections 2.1–2.12] for a comparable treatment in Agda. Finally, see [AG26] for a much more comprehensive treatment of type theory as a syntactic formal system.

1. Types and terms

Three judgements, all relative to a context (= list of typed variables, generically written Γ):

1. Something is a type ($\Gamma \vdash A \text{ Type}$)
2. We have a term of a type ($\Gamma \vdash a : A$)
3. Some terms/types are **judgementally/definitionally** (“computationally”) equal ($\Gamma \vdash a \equiv b$ — for this the terms a and b should be of the same type).

There are “standard” structural rules, e.g. weakening, that we will not discuss. **Our goal here is only to get a good enough understanding of the formal rules that allows us to work informally with the type theory with reasonable confidence, not to study the formal system itself.**

For this reason, I will also permit myself to be sloppy when it comes to substitution.

Martin-Löf introduced primitive type formers, all governed by four kinds of rules.

1. **Formation**: how to make the type.
2. **Introduction**: how to make terms of the type.
3. **Elimination**: how to make *use* of terms of the type.
4. **Computation**: how terms of the type compute / how the elimination and introduction rules are related.

WARNING: In what follows, I will **at first** present an *oversimplification* of the elimination rules, namely a *nondependent* version.

First example: product types

Intuition: think of the cartesian product of two sets.

The rules are similar to those of logical conjunction, but in logical systems, we usually only care about provability. The types play an analogous role to formulas here, but the fundamental difference here is that we really are interested in the resulting terms. Therefore, sometimes people say we have a proof-relevant system. I think it’s best to follow the logical analogy only up to a point. We are just introducing set/type formers axiomatically.

Formation: Given two types A and B , we have a new type, written $A \times B$.

More formally,

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A \times B \text{ Type}}$$

Introduction: Given two terms $a : A$ and $b : B$, we have a new term $\text{pair}(a, b)$ of type $A \times B$.

More formally,

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \text{pair}(a, b) : A \times B}$$

Elimination (preliminary): If we can produce a term of some type C given $a : A$ and $b : B$, then we can produce a term of C given $p : A \times B$.

More formally,

$$\frac{\Gamma, x : A, y : B \vdash c(x, y) : C \quad \Gamma \vdash p : A \times B}{\Gamma \vdash \times\text{-elim}(c, p) : C}$$

Computation (preliminary): In the setting of above rule, the computation rule says that $\times\text{-elim}(c, \text{pair}(a, b))$ and $c[a/x, b/y]$ are equal. (Here $[a/x]$ denotes the substitution of the term a for the variable x .)

More formally,

$$\frac{\Gamma, x : A, y : B \vdash c(x, y) : C \quad \Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \times\text{-elim}(c, \text{pair}(a, b)) \equiv c[a/x, b/y]}$$

Example: In the elimination rule, we can take C to be A and since we have $\Gamma, x : A, y : B \vdash x : A$, we get

$$\frac{\Gamma, x : A, y : B \vdash x : A \quad \Gamma \vdash p : A \times B}{\Gamma \vdash \times\text{-elim}(x, p) : A}$$

with the computation rule informing us that $\times\text{-elim}(x, \text{pair}(a, b)) \equiv a$, so that the term acts as a first projection.

Naturally, we can do something similar for B and the second projection.

Notation. When working informally, we will simply write (a, b) instead of $\text{pair}(a, b)$.

Second example: coproduct types

Intuition: think of the **disjoint** union of two sets / logical disjunction (with some caveats, cf. the discussion logic in the next lecture).

Formation: Given two types A and B , we have a new type, written $A + B$.

More formally,

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A + B \text{ Type}}$$

Introduction: Given a term $a : A$, we have a new term, written $\text{inl}(a)$ and pronounced “in left a”, of type $A + B$, and similarly when we are given $b : B$.

More formally,

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B} \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}$$

Elimination (preliminary): If we can produce a term of some type C whenever we are given $a : A$ or $b : B$, then we can produce a term of C given $p : A + B$.

More formally,

$$\frac{\Gamma, x : A \vdash c(x) : C \quad \Gamma, y : B \vdash d(y) : C \quad \Gamma \vdash p : A + B}{\Gamma \vdash +\text{-elim}(c, d, p) : C}$$

Computation (preliminary): In the setting of above rule, the computation rule says that $+\text{-elim}$ acts “by cases”, e.g. $+\text{-elim}(c, d, \text{inl}(a))$ and $c[a/x]$ are equal.

More formally,

$$\frac{\Gamma, x : A \vdash c(x) : C \quad \Gamma, y : B \vdash d(y) : C \quad \Gamma \vdash a : A}{\Gamma \vdash +\text{-elim}(c, d, \text{inl}(a)) \equiv c[a/x]}$$

$$\frac{\Gamma, x : A \vdash c(x) : C \quad \Gamma, y : B \vdash d(y) : C \quad \Gamma \vdash b : B}{\Gamma \vdash +\text{-elim}(c, d, \text{inr}(b)) \equiv d[b/y]}$$

NB. We refer to the terms given in introduction rules, like `inl` and `inr` as **constructors**.

Function types

We similarly postulate that, for types A and B , there is a new type $A \rightarrow B$ that is (supposed to capture) the type of functions from A to B . If in context $\Gamma, a : A$, we can produce $b_a : B$, then we can construct a function $f : A \rightarrow B$, usually written $\lambda x. b$. The elimination rule says that such functions can be applied, written as $f(x)$ as usual although formally there is a new term $\text{app}(f, x)$ (cf. $\text{pair}(a, b)$), with the computation rule expressing that the functions behave as expected, e.g. in our example, $f(a) \equiv b_a$.

For formal rules, I refer to the more general dependent function types below.

Unlike in set theory, functions are primitive in type theory (strictly speaking, dependent functions are, but that's not so relevant for this point).

2. Dependent types

For mathematics, but also in programming, we want a more expressive system than we have seen so far.

We want **dependent types**: $\Gamma, x : A \vdash B(x)$ Type where the expression $B(x)$ is allowed to depend on x .

For instance, for some fixed type A , we may wish to consider a type of lists over A of some fixed length n . Such a list is typically called a **vector** of length/size n . We will soon introduce a type $\mathbb{N}$ of natural numbers, so that would consider a dependent type $n : \mathbb{N} \vdash \text{Vec}_A(n)$ Type whose terms are vectors (of A) of length n .

We will later see how to encode logic in type theory. The formula expressing that addition on natural numbers is commutative, $\forall(n, m \in \mathbb{N}), n + m = m + n$, will correspond to a type and the expression $n + m = m + n$ will be a dependent type. This is just an instance of the general fact that identity types are dependent types.

The real elimination rules

In line with our adoption of dependent types, the elimination rules given above should all be updated/corrected to account for the fact that C may be a dependent type. For example, for the coproduct types, the elimination rule becomes:

$$\frac{\Gamma, z : A + B \vdash C(z) \text{ Type} \quad \Gamma, x : A \vdash c(x) : C[\text{inl}(x)/z] \quad \Gamma, y : B \vdash d(y) : C[\text{inr}(y)/z] \quad \Gamma \vdash p : A + B}{\Gamma \vdash +\text{-elim}(c, d, p) : C[p/z]}$$

NB. Note how the elimination rules essentially say that we get a term of C anywhere if we can do it at the constructors. **This is the general pattern for all type formers.**

Dependent functions

We now introduce a type of **dependent functions**. Given a type A and a dependent type $B(x)$ over A , we have a type $\prod(x : A), B(x)$ of dependent functions.

We recover nondependent/ordinary functions when $B(x)$ is just some fixed type B , independent of x .

For example, if we have some $a_0 : A$, the function that gives, for every $n : \mathbb{N}$, the vector of n -many copies of a_0 is a dependent function of type $\prod(n : \mathbb{N}), \text{Vec}_A(n)$.

Given $f : \prod(x : A), B(x)$ and $a : A$, we can apply f to a , to obtain $f(a) : B(a)$.

We will later interpret $\forall$ as $\prod$.

Formation	$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma, x : A \vdash B(x) \text{ Type}}{\Gamma \vdash \prod(x : A), B(x) \text{ Type}}$	Introduction	$\frac{\Gamma, x : A \vdash b(x) : B(x)}{\Gamma \vdash \lambda x. b(x) : \prod(x : A), B(x) \text{ Type}}$
Elimination	$\frac{\Gamma \vdash f : \prod(x : A), B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash \text{app}(f, a) : B[a/x]}$	Computation	$\frac{\Gamma, x : A \vdash b(x) : B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash \text{app}(\lambda x. b(x), a) \equiv b[a/x]}$

Notation. We will just write $f(a)$ instead of $\text{app}(f, a)$ when working informally.

Dependent pairs

We now introduce a type of **dependent pairs**. Given a type A and a dependent type $B(x)$ over A , we have a type $\sum(x : A), B(x)$ of dependent pairs.

For example, the type of vectors of any (finite) length would be $\sum(n : \mathbb{N}), \text{Vec}_A(n)$.

This type generalizes the product type: when $B(x)$ does not depend on $x : A$, we recover the ordinary product $A \times B$.

Given $a : A$ and $b : B(a)$, we can form $(a, b) : \sum(x : A), B(x)$.

Conversely, given $p : \sum(x : A), B(x)$, we get projections $\text{pr}_1(p) : A$ and $\text{pr}_2(p) : B(\text{pr}_1(p))$.

We will later interpret $\exists$ as a *refinement* of $\sum$.

Formation	$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma, x : A \vdash B(x) \text{ Type}}{\Gamma \vdash \sum(x : A), B(x) \text{ Type}}$	Introduction	$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \text{pair}(a, b) : \sum(x : A), B(x) \text{ Type}}$
------------------	--	---------------------	---

Elimination

$$\frac{\Gamma, z : \sum(x : A), B(x) \vdash C(z) \text{ Type} \quad \Gamma, x : A, y : B(x) \vdash c : C[\text{pair}(x, y)/z]}{\Gamma \vdash p : \sum(x : A), B(x) \quad \Gamma \vdash \sum\text{-elim}(c, p) : C[p/z]}$$

Computation

$$\frac{\Gamma, z : \sum(x : A), B(x) \vdash C(z) \text{ Type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x] \quad \Gamma \vdash c : C[\text{pair}(a, b)/z]}{\Gamma \vdash \sum\text{-elim}(c, \text{pair}(a, b)) \equiv c}$$

Notation. We will just write (a, b) instead of $\text{pair}(a, b)$ when working informally.

3. Getting off the ground

So far, all our type formers can only be used when we already have some types. Hence, we will need some types to get off the ground. Let's introduce some now.

The unit type

We introduce a type with exactly one element.

Formation	$\frac{}{\Gamma \vdash \mathbb{1} \text{ Type}}$	Introduction	$\frac{}{\Gamma \vdash \star : \mathbb{1}}$
------------------	--	---------------------	---

$$\text{Elimination} \quad \frac{\Gamma, x : \mathbb{1} \vdash A(x) \text{ Type} \quad \Gamma \vdash a : A[\star/x] \quad \Gamma \vdash u : \mathbb{1}}{\Gamma \vdash \mathbb{1}\text{-elim}(a, u) : A[u/x]}$$

with the **computation** rule saying that $\mathbb{1}\text{-elim}(a, \star) \equiv a$.

The booleans / two-element type

We define $\mathbb{2} := \mathbb{1} + \mathbb{1}$ to be the type with exactly two elements, that we will call 0 and 1 (formally these are $\text{inl}(\star)$ and $\text{inr}(\star)$).

The empty type

We introduce a type with no elements at all.

Formation $\frac{}{\Gamma \vdash \emptyset \text{ Type}}$ **Introduction** No elements, so no introduction rule!

$$\text{Elimination} \quad \frac{\Gamma, x : \emptyset \vdash A(x) \text{ Type} \quad \Gamma \vdash u : \emptyset}{\Gamma \vdash \emptyset\text{-elim}(u) : A[u/x]} \text{ with no computation rule.}$$

Note that the elimination rule is like *ex-falso*(-quod-libet) in logic.

The natural numbers

The type is inductively generated by 0 and successors.

$$\text{Formation} \quad \frac{}{\Gamma \vdash \mathbb{N} \text{ Type}} \quad \text{Introduction} \quad \frac{}{\Gamma \vdash 0 : \mathbb{N}} \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}$$

Elimination This is like the principle of induction:

$$\frac{\Gamma, x : \mathbb{N} \vdash A(x) \text{ Type} \quad \Gamma \vdash u : A[0/x] \quad \Gamma, y : \mathbb{N}, z : A[m/x] \vdash v(y, z) : A[\text{succ}(y)/x] \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \mathbb{N}\text{-elim}(u, v, n) : A[n/x]}$$

The **computation** rule says that $\mathbb{N}\text{-elim}(u, v, 0) \equiv u$ and given $w : A[m/x]$, we have $\mathbb{N}\text{-elim}(u, v, \text{succ}(m)) \equiv v[m/y, \mathbb{N}\text{-elim}(u, v, m)/z]$, i.e. the eliminator computes recursively.

Using the elimination principle and function types, we can define a term $\text{add} : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$ with the computation rules ensuring that

$$\text{add}(n, 0) \equiv n \quad \text{and} \quad \text{add}(n, \text{succ}(m)) \equiv \text{succ}(\text{add}(n, m)).$$

From now on, we will write $n + 1$ instead of the more formal term $\text{succ}(n)$. (This is justified by the above since we define 1 as $\text{succ}(0)$).

4. Universes

To define various types of mathematical structures, e.g. the type of (small) groups, we will need universes. A **universe** is “a type of types”. Of course, we must be careful to avoid Russell-style paradoxes, so a universe is like a type of *small* types. One then typically assumes a hierarchy of universe $\mathcal{U}_0, \mathcal{U}_1, \dots$ with $\mathcal{U}_i$ living in $\mathcal{U}_{i+1}$.

We ask that universes are closed under all type formers, so that e.g. if $A, B : \mathcal{U}_i$, then $A \times B : \mathcal{U}_i$. More generally, for $A : \mathcal{U}_i$ and $B : \mathcal{U}_j$, we postulate that $A \times B : \mathcal{U}_{\max(i,j)}$.

To be more precise and to preserve the distinction between terms and types, technically the terms of a universe are codes for types with a decoding function usually called “el”, i.e. formally we add rules

$$\frac{}{\Gamma \vdash \mathcal{U}_i \text{ Type}} \quad \frac{\Gamma \vdash c : \mathcal{U}_i}{\Gamma \vdash \text{el}(c) \text{ Type}}$$

and ask that we have appropriate codes for types built using type formers. Having said this, we will just write $A : \mathcal{U}_i$ treating it as a convenient abbreviation for what is technically a more formal statement involving codes.

5. Example of encoding mathematics

In mathematics, a **monoid** is a set M with a binary “multiplication” operation $(-) \cdot (-) : M \times M \rightarrow M$ and an element $e \in M$ such that the multiplication is associative, i.e. $(x \cdot y) \cdot z = x \cdot (y \cdot z)$ holds for all $x, y, z \in M$ and e is neutral, i.e. $x \cdot e = x$ and $e \cdot x = x$ for every $x \in M$.

We will soon introduce identity types, written Id , which allow us to encode the above in type theory, as follows. The type Monoid of monoids (in a universe $\mathcal{U}$) is:

$$\begin{aligned} \text{Monoid} := & \sum (M : \mathcal{U}), \sum (f : M \times M \rightarrow M), \sum (e : M), \\ & \left(\prod (x, y, z : M), \text{Id}_M(f(f(x, y), z), f(x, f(y, z))) \right) \times \\ & \left(\prod (x : M), \text{Id}_M(f(x, e), x) \right) \times \left(\prod (x : M), \text{Id}_M(f(e, x), x) \right) \end{aligned}$$

Here we abbreviated $\prod(x : M), \prod(y : M), \prod(z : M) \dots$ for readability.

Further allowing ourselves to write infix notation for the multiplication and $x = y$ instead of $\text{Id}_M(x, y)$, we get

$$\begin{aligned} \text{Monoid} := & \sum (M : \mathcal{U}), \sum (- \cdot - : M \times M \rightarrow M), \sum (e : M), \\ & \left(\prod (x, y, z : M), (x \cdot y) \cdot z = x \cdot (y \cdot z) \right) \times \\ & \left(\prod (x : M), x \cdot e = x \right) \times \left(\prod (x : M), e \cdot x = x \right) \end{aligned}$$

Notice the use of dependent types!

A term of this type is a tuple $(M, \cdot, e, \alpha, \beta, \gamma)$, with α a proof that $\cdot$ is associative, etc.

Actually, a proper translation requires M to be a set in the homotopy type theory sense, but we will only discuss this point later.

6. Identity types

The identity type is an **inductively generated family**: it depends on two terms of a type.

$$\text{Formation} \quad \frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id}_A(a, b) \text{ Type}}$$

$$\text{Introduction} \quad \frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash a : A}{\Gamma \vdash \text{refl}(a) : \text{Id}_A(a, a)}$$

Elimination We follow the pattern of the other elimination rules which roughly say that “it is enough to do something for constructors/things coming from the introduction rule(s)”.

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma, x : A, y : A, p : \text{Id}_A(x, y) \vdash B(x, y, p) \text{ Type} \quad \Gamma, x : A \vdash r(x) : B[x/x, x/y, \text{refl}(x)/p] \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash q : \text{Id}_A(a, b)}{\Gamma \vdash J(r, a, b, q) : B[a/x, b/y, q/p]}$$

The name J instead of Id-elim is traditional. It seems Martin-Löf simply picked the next letter after I (for identity types), and in turn, Streicher picked the next letter after J for his Axiom K .

Following the HoTT Book, the elimination rule is also called **path induction**.

The computation rule says that J reduces to r (when it makes sense).

Computation

$$\frac{\begin{array}{c} \Gamma \vdash A \text{ Type} \quad \Gamma, x : A, y : A, p : \text{Id}_A(x, y) \vdash B(x, y, p) \text{ Type} \\ \Gamma, x : A \vdash r(x) : B[x/x, x/y, \text{refl}(x)/p] \quad \Gamma \vdash a : A \end{array}}{\Gamma \vdash J(r, a, a, \text{refl}(a)) \equiv r(a)}$$

NB. Rijke introduces unary identity types as primitives, whereas we used binary ones. They are interderivable.

Inverting and composing identities

To illustrate the power of the elimination rule, we briefly sketch how it lets us invert and compose identities.

Suppose we are in a context Γ where we have a type A , terms $a, b : A$ and $q : \text{Id}_A(a, b)$. Taking $B(x, y, p)$ to be $\text{Id}_A(y, x)$, we get an element $J(a, b, q) : \text{Id}_A(b, a)$, that we call q^{-1} , as soon as we give $r(x) : \text{Id}_A(x, x) \equiv B(x, x, \text{refl}(x))$. But we can simply pick $r(x) \equiv \text{refl}(x)$. This gives rise to a function $(-)^{-1} : \text{Id}_A(a, b) \rightarrow \text{Id}_A(b, a)$ with the computation rule giving us $\text{refl}(a)^{-1} \equiv \text{refl}(a)$.

Similarly, we can derive a function $(-) \bullet (-) : \text{Id}_A(a, b) \rightarrow \text{Id}_A(b, c) \rightarrow \text{Id}_A(a, c)$, essentially by setting $p \bullet \text{refl}(b) \equiv p$ for $p : \text{Id}_A(a, b)$.

Isn't refl the only term?! Well, yes, but mostly no...

At first (and probably at second and third) glance, the rules of the identity type may suggest that refl is the only term, i.e. that the elimination rule would allow us to prove something like “every $p : \text{Id}_A(a, b)$ must be identified with $\text{refl}(a)$ ”.

This is not the case. In fact, the statement in quotes does not even type-check, because $\text{refl}(a) : \text{Id}_A(a, a)$, while $p : \text{Id}_A(a, b)$.

But what about the statement that “every $p : \text{Id}_A(a, a)$ must be identified with $\text{refl}(a)$ ”?

(Note that this is Streicher’s Axiom *K*.)

This is subtler, but the point is that the dependent type $B(x, y, p)$ in the elimination rule depends on a *generic* term $p : \text{Id}_A(x, y)$. We cannot assume that $x \equiv y$. So that statement is also **not** provable using the elimination rule and in fact it is not provable at all (cf. the Hofmann–Streicher groupoid model).

On the other hand, we can use the elimination rule to prove that for $p : \text{Id}_A(a, b)$, the terms (a, b, p) and $(a, a, \text{refl}(a))$ of type $\sum(x : A), \sum(y : A), \text{Id}_A(x, y)$ are identified.

Slogan: “As a family, Id_A does have a single element.”

This fact is witnessed in type theory by the equivalences (a concept to be introduced later)

$$\sum(x : A), \sum(y : A), \text{Id}_A(x, y) \simeq A \quad \text{and} \quad \sum(y : A), \text{Id}_A(a, y) \simeq \mathbb{1}$$

that we will revisit.

On the other hand, $\text{Id}_A(a, a)$ can have many elements, e.g. for A the circle and a its basepoint, we will see that the identity type is equivalent to the type of integers!

Notation and terminology

From now on, we will usually write $a = b$ instead of $\text{Id}_A(a, b)$, sometimes adding a subscript to the $=$ -sign for the type when it’s not obvious from the context.

We will also write refl instead of $\text{refl}(a)$ whenever it’s clear at which element we are applying it.

Following the HoTT perspective, we will call elements of identity types **paths** or **identifications** instead of equalities.

Type families and functions acting on identifications

Every function acts on identifications: given $f : A \rightarrow B$, we have a function

$$\text{ap}_f : \prod (a_1, a_2 : A), a_1 = a_2 \rightarrow f(a_1) = f(a_2)$$

defined via $\text{ap}_f(\text{refl}) := \text{refl}$ (where we've omitted the first two arguments).

The letters “ap” stand for “action on paths”.

Given a dependent type $B(x)$ over A , we have a function

$$\text{transport}^B : \prod (a_1, a_2 : A), B(a_1) \rightarrow a_1 = a_2 \rightarrow B(a_2)$$

defined via $\text{transport}^B(b, \text{refl}) := b$ (where we've omitted the first two arguments again, and written the other arguments as a pair).

7. Using the elimination rules informally

Just as we've been doing for the identity types, we will use all elimination rules **informally**. For instance, we define the swap function

$$\text{swap} : A + B \rightarrow B + A$$

by just writing the clauses

$$\text{inl}(a) \mapsto \text{inr}(a) \quad \text{and} \quad \text{inr}(b) \mapsto \text{inl}(b)$$

and we don't bother writing out formal derivations.

[This practice is mirrored in Agda via pattern matching.](#)